Raport de Laborator

Lucrare: Depanare Script Python și Gestiune Versiuni cu Git
Student: Gaju Daniel
Fișier analizat: inventar.py
Asistent: Reitman Patricia

Obiectivul lucrării: Identificarea, diagnosticarea și corectarea erorilor logice, de sintaxă și de execuție dintr-un script Python predefinit. Orice modificare a fost izolată într-un commit individual în sistemul de control al versiunilor (Git), păstrând un istoric curat și descriptiv.

1. Erorile și corecția lor

Execuția scriptului s-a oprit succesiv la următoarele erori, care au fost remediate luând codul linie cu linie:

`SyntaxError: invalid syntax (Linia 8)`
Problemă: Lipsea caracterul două puncte (`:`) obligatoriu la definirea unei funcții în Python.
Soluție aplicată: Funcția a fost corectată din `def incarca(cale)` în `def incarca(cale):`.

```
 8  def incarca(cale):
 9      """Citeste lista de tinte din fisierul JSON."""
10      with open(cale) as f:
11          return json.load(f)
```

Fig. 1.1

`IndentationError: expected an indented block (Linia 19)`
Problemă: Instrucțiunea `return None` nu respecta alinierea la dreapta (indentarea) necesară pentru a fi recunoscută în interiorul blocului de cod.
Soluție aplicată: S-au adăugat 4 spații în fața instrucțiunii pentru o indentare corectă.

```
14  def port_valid(text):
15      """Intoarce portul ca int, sau None daca nu e un num
16      try:
17          return int(text)
18      except ValueError:
19          return None
```

Fig. 1.2

`NameError / TypeError (Linia 52)`
Problemă: Tentativa de a apela variabila inexistentă `tinta` (în loc de `tinte`) și de a concatena direct un șir de caractere cu o valoare întreagă (returnată de `len()`).
Soluție aplicată: S-a corectat numele variabilei și s-a aplicat funcția de conversie în șir de caractere: `print("Tinte citite: " + str(len(tinte)))`.

```
46  def main():
47      p = argparse.ArgumentParser()
48      p.add_argument("fisier")
49      p.add_argument("--prag", default=1024)
50      a = p.parse_args()
51
52      tinte = incarca(a.fisier)
53      print("Tinte citite: " + str(len(tinte)))
54
```

Fig. 1.3

`KeyError: 'serviciu' (Linia 26)`
Problemă: Accesarea directă a unei chei lipsă din dicționarul JSON a provocat oprirea subită a programului.
Soluție aplicată: Accesarea `t["serviciu"]` a fost înlocuită cu metoda sigură `t.get("serviciu", "necunoscut")`, permițând codului să continue cu o valoare de rezervă (fallback).

```
22  def pe_servicii(tinte):
23      """Dict: serviciu -> lista de porturi."""
24      grupe = {}
25      for t in tinte:
26          serviciu = t.get("serviciu", "necunoscut")
27          port = port_valid(t["port"])
```

Fig. 1.4

`AttributeError: 'NoneType' object has no attribute 'items' (Linia 55)`
Problemă: Funcția `pe_servicii()` nu avea o instrucțiune `return`, întorcând implicit `None`. Astfel, programul încerca să folosească o metodă de dicționar (`.items()`) pe un obiect nul.
Soluție aplicată: S-a adăugat instrucțiunea `return grupe` la finalul (linia 33) funcției `pe_servicii()`.

```
31          grupe[serviciu] = []
32          grupe[serviciu].append(port)
33          return grupe
34
```

Fig. 1.5

`TypeError: '>' not supported... (Linia 41)`
Problemă: Compararea matematică a unei variabile de tip întreg (`port`) cu una de tip text (`prag` citit din argumentele liniei de comandă).
Soluție aplicată: Conversia variabilei `prag` la număr întreg în momentul comparării: `if port > int(prag):`.

```
36  def peste_prag(tinte, prag):
37      """Lista gazdelor cu un port mai mare decat pragul."
38      rezultat = []
39      for t in tinte:
40          port = int(t["port"])
41          if port > int(prag):
42              rezultat.append(t["gazda"])
43          return rezultat
```

Fig. 1.6

2. Gestiunea Versiunilor (Git)

Pe parcursul procesului de depanare s-a aplicat metodologia de versionare strictă:

Acțiune realizată	Comenzi utilizate (Exemple)
Commit-uri izolate	`git add inventar.py` `git commit -m "Mesaj descriptiv"`
Corectarea istoricului (Amend)	`git commit --amend --no-edit` *Utilizat pentru a corecta erori introduse pe parcurs fără a polua istoricul.*
Sincronizare (Push Force)	`git push --force` *Utilizat pentru a suprascrie istoricul de pe repository-ul remote cu istoricul reparat și curățat local.*

3. Rezultatul Final

În urma aplicării corecțiilor, programul rulează cu succes procesând inputul complet: sare peste porturile invalide (ex: `abc`), grupează serviciile (`ssh`, `http`, `mysql`, `necunoscut`) și creează la final raportul operațional `raport_inventar.json`.

```
Peste 1000: []
Raport salvat in raport_inventar.json
[mac@MacBook-Air depanare % git log --oneline                                ]
458268f (HEAD -> main) Fix TypeError: convertit prag in int la linia 41
131d888 Fix AttributeError: adaugat return la finalul functiei pe_servicii
5c700b2 Fix KeyError: inlocuit t['serviciu'] cu t.get() la linia 26
77dfe9b Fix NameError: corectat tinta in tinte si adaugat str() la linia 52
8ce5eb7 Fix IndentationError la linia 19
e5eb060 Fix SyntaxError: adaugat colon la def incarca linia 8
402a416 Fix IndentationError la return None
801fc96 Fix SyntaxError: adaugat colon la def incarca
f4413c8 Fix SyntaxError: adaugat colon la def incarca
caad7a0 (origin/main, origin/HEAD) Ignor mediul virtual
db96fcf Update README.md with new content
f0819d1 Adaug salut.py
343d19d Initial commit
[mac@MacBook-Air depanare % git push                                         ]
To https://github.com/gajufcim-arch/exercitiu-git.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/gajufcim-arch/exercitiu-g
it.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
```